第7章 第6节 Prompts 详解

- 来源: https://class.imooc.com/lesson/5280#mid=104935
- 阶段: 阶段三: 【工具与协议】让AI连接世界
- 周: 第7周 MCP：模型上下文协议
- 模块: Tools、Prompts详解
- 课节: 图文 1-2 Prompts 详解

1-2 Prompts 详解

第7章 第6节 Prompts 详解

> [!TIP] 阅读指南 上一节学习了 Tools，让 AI 能够执行动态操作。本节学习 MCP 的第三大核心能力：Prompts（预制的使用模板），它能把常见的操作流程标准化。

6.1 Prompts：预制的使用模板

Prompts 要解决什么问题

Resources 和 Tools 都学会了，但用户每次都要"从零开始"描述需求：

场景：数据分析需求
━━━━━━━━━━━━━━━━

用户A："帮我分析2024年的订单趋势，按月统计金额，生成图表描述"
用户B："能不能看下去年每个月的订单金额变化？"
用户C："分析订单数据，时间维度是月份，指标是金额总和"

问题：
× 三个用户其实想要同样的分析
× 但表述不同，AI 可能理解有偏差
× 每次都要完整描述需求，很繁琐

1
2
3
4
5
6
7
8
9
10
11

预览

Prompts 就是为了标准化常见的使用场景。

Prompts 就是为了标准化常见的使用场景

Prompts 的工作方式

```
Prompts = 预制的 Prompt 模板 + 参数

示例：
——————————————————
Prompt 名称：monthly_order_analysis
模板内容：
  "分析 {year} 年的订单数据，按月统计 {metric} 指标，
   生成趋势描述和关键发现"

参数：
  - year: 年份
  - metric: 指标（金额/数量/用户数）

使用：
  → 用户只需选择这个 Prompt
  → 填写参数：year=2024, metric=金额
  → AI 自动生成完整的分析流程
```

Prompts 的设计示例

示例：天气查询 MCP Server 的 Prompts（伪代码）

```
# 注意：以下为伪代码示意，不可直接运行

class WeatherMCPServer:
    def list_prompts(self):
        # ▸ 返回所有可用的 Prompts
        return [
            {
                "name": "daily_weather_summary",
                "description": "生成指定城市的每日天气摘要",
                "arguments": [
                    {
                        "name": "city",
                        "description": "城市名称",
                        "required": True
                    },
                    {
                        "name": "date",
                        "description": "日期（默认今天）",
                        "required": False
                    }
                ]
            },
            {
                "name": "weather_travel_advice",
                "description": "基于天气给出旅行建议",
                "arguments": [
                    {
                        "name": "city",
                        "description": "目的地城市",
                        "required": True
                    },
                    {
                        "name": "days",
                        "description": "旅行天数",
                        "required": True
                    }
                ]
            }
        ]

    def get_prompt(self, name, arguments):
        # ▸ 生成具体的 Prompt 内容
        if name == "daily_weather_summary":
```

```
            city = arguments["city"]
            date = arguments.get("date", "今天")

            return {
                "messages": [
                    {
                        "role": "user",
                        "content": f"""
请为{city}生成{date}的天气摘要报告，包括：
1. 温度范围和体感温度
2. 天气状况（晴/雨/阴等）
3. 空气质量指数
4. 出行建议（是否需要带伞、注意防晒等）

请使用温暖友好的语气，就像天气预报员一样。
                        """
                    }
                ]
            }

        elif name == "weather_travel_advice":
            city = arguments["city"]
            days = arguments["days"]

            return {
                "messages": [
                    {
                        "role": "user",
                        "content": f"""
我计划去{city}旅行{days}天。
请根据未来{days}天的天气预报，给我以下建议：
1. 最佳出行时间段
2. 需要准备的衣物和装备
3. 天气可能影响的活动安排
4. 特别需要注意的事项

请结合天气数据给出具体、实用的建议。
                        """
                    }
                ]
            }
```

使用流程

```
方式1：传统方式（用户自己写）
━━━━━━━━━━━━━━━━━━━━━━━━

用户："帮我看看上海今天的天气，告诉我要不要带伞"
  ↓
AI 自己理解需求，查询天气，生成回复

方式2：使用 Prompt 模板（标准化）
━━━━━━━━━━━━━━━━━━━━━━━━━━━

用户：选择 "daily_weather_summary" Prompt
     填参数：city="上海", date="今天"
  ↓
1. AI 应用调用 get_prompt("daily_weather_summary", {city:"上海"})
2. Server 返回完整的 Prompt 模板
3. AI 使用这个模板，查询天气，生成标准化的摘要
```

使用 Prompt 模板不仅能确保输出格式的一致性（总是包含温度、空气质量等关键信息），还能统一 AI 的语气（如维持温暖友好的播报风格），并显著提升用户体验，因为用户不再需要绞尽脑汁描述复杂的查询需求。

Prompts 的两种使用模式

可能会有人疑问：AI的优势不就是理解自然语言吗？为什么还要让用户填表单选模板？

AI的优势不就是理解自然语言吗？为什么还要让用户填表单选模板？

实际上，Prompts 有两种不同的使用方式：

模式1：AI背后自动调用（对用户透明）

```
用户："我要退货，订单号 12345"
  ↓
AI应用智能分析：
  1. 识别意图 → 这是退货需求
  2. 提取参数 → order_id=12345
  3. 自动选择 "handle_refund_request" Prompt
  4. 自动填充参数
  ↓
执行退款流程（用户感觉在自然对话）

优势：
✓ 用户无感知，纯自然语言交互
✓ AI自动选择最合适的Prompt
✓ 对话体验流畅

适用场景：
→ 日常查询、咨询类需求
→ 对话式客服
→ 智能助手
```

模式2：用户主动选择（显式交互）

```
用户：点击 "退款处理" 按钮
  ↓
系统弹出表单：
  +-------------------------
  | 退款处理流程
  +-------------------------
  | 订单号：[12345____]
  | 退货原因：[质量问题▼]
  | 退款方式：[原路退回▼]
  |
  |      [取消]  [提交]
  +-------------------------
  ↓
执行退款流程

优势：
✓ 用户明确知道在做什么
✓ 可以修改、确认参数
✓ 更可控，适合关键操作

适用场景：
→ 敏感操作（退款、删除数据）
→ 需要用户确认的流程
→ 批量操作、复杂表单
```

两种模式的选择依据

维度	模式1（AI自动）	模式2（用户选择）
交互方式	纯对话	点击+填表单
用户体验	更自然流畅	更明确可控
适用操作	查询、咨询	敏感、关键操作
错误风险	AI可能误判	用户手动确认
实现复杂度	需要NLU能力	相对简单

> NLU(Natural Language Understanding)，自然语言理解的缩写

实际项目中往往是混合使用。比如一个智能客服系统：

```
智能客服系统示例：
━━━━━━━━━━━━━━━━

场景1：简单查询（模式1）
  用户："我的订单到哪儿了？"
  → AI自动识别，调用"order_inquiry" Prompt，直接回答

场景2：敏感操作（模式2）
  用户："我要退款"
  AI："好的，请确认以下信息："
  → 弹出退款表单，用户填写确认后执行

场景3：复杂需求（混合）
  用户："帮我分析一下最近的销售情况"
  AI："我找到了几个分析模板，您需要哪种？"
  → 推荐Prompt列表，用户选择后填参数
```

6.2 下一节预告

学会了 Resources、Tools、Prompts 各自的用途后，下一节将把它们三者组合起来，通过一个完整的智能客服系统场景，展示 MCP 三大能力如何协同工作，搞定真实世界中的复杂业务需求。

6.3 ■ 学点英语

中文	English	音标	说明
预制模板	Predefined Template	/ˌpriːdɪˈfaɪnd ˈtemplət/	将常见操作流程标准化的预设Prompt模式
占位符	Placeholder	/ˈpleɪshoʊldər/	Prompt模板中预留的变量位置{{}}，用户传入具体参数
标准化场景	Standardized Scenario	/ˈstændərdaɪzd sɪˈnærioʊ/	用模板统一常见需求的处理方式和输出格式
模板参数填充	Template Parameter Binding	/ˈtemplət pəˈræmɪtər ˈbaɪndɪŋ/	用户传入参数替换模板中占位符的过程